2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW) | 979-8-3503-0208-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/CCGridW59191.2023.00037

Malware Detection using API Calls Visualisations and Convolutional Neural Networks

Jaime Pizarro Barona
Facultad de Ingeniería en Electricidad y Computación
Escuela Superior Politécnica del Litoral
Guayaquil, Ecuador
japibaro@espol.edu.ec

Joseph Avila Alvarez
Facultad de Ingeniería en Electricidad y Computación
Escuela Superior Politécnica del Litoral
Guayaquil, Ecuador
josdavil@espol.edu.ec

Carlos Jiménez Farfán
Facultad de Ingeniería en Electricidad y Computación
Escuela Superior Politécnica del Litoral
Guayaquil, Ecuador
cajifarf@espol.edu.ec

Joangie Márquez Aguilar
Facultad de Ingeniería en Electricidad y Computación
Escuela Superior Politécnica del Litoral
Guayaquil, Ecuador
jomamarq@espol.edu.ec

Rafael I. Bonilla
Facultad de Ingeniería sisen Electricidad y Computación
Escuela Superior Politécnica del Litoral
Guayaquil, Ecuador
rabonilla@espol.edu.ec

***Abstract*—This research explores and analyzes different API Calls sequence transformation methods into images to train deep learning models and determine which combination of these methods and models performs better. We generated images from API Calls sequences using Simhash and FreqSeq. The results were compared by training two well-known Convolutional Network architectures (ResNet50v2 and MobileNetv2). This work presents our experience running these experiments highlighting the results obtained and the challenges we faced.**

***Index Terms*—malware detection, API calls, artificial intelligence, deep learning, convolutional neural networks**

I. Introduction

The increment of remote jobs due to the pandemic has led to an increase in cybercrime of 148% in 2020 and 2021. [1] Lallie et al. presented a cyberattack analysis of the first quarter of 2020. These cyberattacks were highly related to the pandemic and corroborated that 65% of the cyberattacks involved malware [2].

Kaspersky published in their annual security bulletin statistical data about malware in Latin America. The cyberthreats leading the 2021 malware statistics were banking Trojans such as Zbot and malware related to phishing [3].

Lallie et al. and Pranggono et al. indicate that ransomware was mainly used to attack institutions that offer health services [2] [4]. Furthermore, institutions in the financial sector have also been targeted.

Cybercriminals use multiple types of malware, increasingly specialized. Among these are metamorphic, polymorphic and fileless. Currently, these types of malware are quite popular and dangerous due to their ability to change their operational codes (opcodes) by rewriting a new version of themselves on the infected host and hiding in main memory without leaving traces on disk [5]. For this reason, most traditional signature-based antivirus or intrusion detection systems (IDS) fail to detect them.

Currently, malware is analyzed to better understand its behavior and counter future threats. There are 3 types of malware analysis: static, dynamic and memory-based analysis. Memory-based analysis is considered one of the most acceptable techniques for determining malware behavior [6]. This analysis focuses on examining the traces left by the malware in the volatile memory of the host, which are independent of the bytes of the malware executable. This analysis allows the malware to be characterized a priori in order to be identified later. One of the descriptors that contain the most useful information about malware behavior are API Calls, since the vast majority of malicious actions require the usage of system APIs, such as the win32 API in Windows [7]. In addition, malware developers often use existing code as the basis for their new malware, so many features like API Calls remain constant [8] [9].

Additionally, research has shown exceptional results in the usage of images for malware identification. A clear example is the usage of a technique called SimHash to obtain a fixed-size binary that can be converted to an image. This technique has already been used with opcodes [10], but there is also the possibility of using it in API Calls. Another example is the transformation of memory dumps into images to train

979-8-3503-0208-0/23/$31.00 ©2023 IEEE
DOI 10.1109/CCGridW59191.2023.00037

Authorized licensed use limited to: ESCUELA POLITECNICA DEL LITORAL (ESPOL). Downloaded on October 30,2023 at 03:27:43 UTC from IEEE Xplore. Restrictions apply.

machine learning models, which have proven to be effective in identifying behavior-based malware [11]. These types of models generally use Convolutional Neural Networks (CNNs) and their variants (ResNet, LeNet, AlexNet, etc.), showing great effectiveness when working with images by obtaining predictions with 96.3% precision [12] [13]. In this paper we propose a method to generate color images that combines the frequency of API Calls and their order in the sequences. This approach adds more meaning to the generated patterns.

This paper is organized as follows. Section II presents the background and related work. Sections III and IV describe the dataset and the methodology followed by our team. Section V includes our results with a proper discussion in Section VI. We finish our work in Section VII with our conclusions and ideas for future work.

II. Background and Related Work

The purpose of this section is to provide context of the different types of malware analysis, emphasizing memory-based analysis. Likewise, the application of Machine Learning and Deep Learning in malware detection is presented, as well as the usage of performance metrics in the models. Finally, the conversion techniques of API Calls to images are presented in this section.

A. Static Analysis

Static analysis is a technique that does not require the execution of malicious executable files in order to perform the analysis. This analysis makes use of properties of executable files such as strings, opcodes, byte sequences, among others [12].

Signature-based malware detection uses static analysis to perform signature extraction and identify malware that matches the extracted signature. The signature is a sequence of bytes that allows a specific malware to be identified [14].

Static analysis is effective against malware whose signature is known. However, this type of analysis is limited by code obfuscation. Modern malware uses obfuscation techniques to avoid detection or analysis, as these techniques change the properties of executable files as well as their signature [15] [16]. Due to this limitation, this scan is not efficient in detecting this kind of malware.

B. Dynamic Analysis

Dynamic analysis is also known as behavioral analysis. For the application of this analysis, it is necessary to execute the malicious files and then inspect the malware based on the characteristics of the behavior that it had in the environment in which it was executed. An example of this type of analysis is the analysis of network behavior in response to the execution of malware [9]. This analysis is usually more effective compared to static analysis because it allows both known and unknown malware to be identified, as it is not affected by obfuscation techniques [14].

C. Memory Based Analysis

Memory-based scanning is recognized by some authors as part of dynamic malware scanning, as it requires running the malware to perform the scan in memory.

This is an automated method that detects new and unknown malware by analyzing a system's crash dumps [14].

This analysis is based on the fact that all executable code and data generated in a system leave traces in memory. Because of this, it identifies the presence of malicious code and data by looking for patterns and suspicious activities in memory dumps. This allows us to decide which features are or are not appropriate for analysis [17].

There are studies that carry out malware classifiers based on particular characteristics of memory dumps, such as opcodes, API calls, memory accesses, among others.

Memory access patterns are used as an alternative to opcodes, since it could be argued that similar opcodes result in similar memory access patterns [18]. Also, from the entire memory dump, only the section of memory that includes the DLLs, thread stack, data segment, and other data could be used [19].

This investigation uses memory-based analysis, in particular, we analyze the sequence of API calls. Extracting features from API Calls has been a popular approach among researchers due to its good performance in detecting malware [6] [11].

D. Classification based on API Calls

API Calls were designed for Windows application developers to interact with the operating system [20].

Certain API Calls appear more frequently in sequences corresponding to malware. One of these API Calls is **GetFileType**. This is closely related to what was found by Sundarkumar et al., where they managed to show how the malware generated suspicious behavior and captured the API Calls that were most executed. From this, they also managed to define "if-then" rules for the different types of malware against benign programs [7].

Furthermore, Ki et al. first categorized the APIs into different classes to generate new sequences in which they included the categories instead of the calls [21]. Also, Tang et al. classified API Calls according to different criteria such as their functionality, malicious intent, among others [9]. The objective of their research was to obtain the frequency of these categories in the execution of malware. Finally, API Calls had also been categorized in order to generate representative graphs of these categories and perform analysis from these graphs [13].

E. Malware visualization techniques

Ni et al. employed an algorithm called SimHash [10]. Simhash uses Hash functions (md5, SHA-256, etc.) to generate a summary of the malware's opcodes and transform it into images of various sizes. Simhash is an encoding technique that, given an input, returns a hash of a fixed size, which will depend on the hash function you use, such as md5, SHA-1,

Authorized licensed use limited to: ESCUELA POLITECNICA DEL LITORAL (ESPOL). Downloaded on October 30,2023 at 03:27:43 UTC from IEEE Xplore. Restrictions apply.

SHA-2, SHA-128, SHA-256. The peculiarity of this technique is that if the inputs are similar, the hashes will also be similar, resulting in similar images [22]. A maximum accuracy of 99,260% and an average of 98,862% was achieved using this technique.

Tang et al. converted API Calls to images and then trained Convolutional Neural Networks (CNN). When comparing the results of using CNN relative to k-NN, the usage of CNN on images showed better performance with an accuracy of 99% [9].

F. Machine Learning vs Deep Learning

Currently, the amount of research carried out on the use of Machine Learning for malware identification is extensive, and memory-based analysis has become quite popular thanks to the number of features it presents [6–10], [12], [13], [18], [19].

For example, Bozkir et al. used Machine Learning algorithms such as *Random Forest*, *XGBoost*, *Sequential Minimal Optimization* (SMO), *Support Vector Machines* (SVMs) and *J48*, obtaining as the best result an accuracy of 94.65% with SMO. Three of these five techniques are tree-based, while two of them are involved in structured learning methods. In this way, they proposed to examine whether the problem domain can be learned by applying tree-based methods and kernel machines [12].

These shallow learning techniques mentioned (SMO, SVMs, Decision Trees, etc.) are limited when working with data that have large amounts of *features* by nature, for example the pixels of an image, or the frames of a video, which have a large amount of information.

Using Deep Learning models such as Deep Neural Networks (DNNs) eliminates the limitations of Machine Learning, since these models allow automatically extracting all the features during their training [23].

G. Deep Learning

There are some types of *Deep Neural Networks* (DNNs) that have been used for malware detection. Among them are *Recurrent Neural Networks* (RNNs), which are generally used when data is represented as time series due to their excellent performance. Li et al. used recurrent neural networks to classify malware from the API Calls obtained from memory based on the sequential-temporal order in which they are generated [11]. On the other hand, *Convolutional Neural Networks* (CNNs) are a type of DNN that has had a great impact when solving object detection and image classification tasks due to their ability for extracting high- and low-level features from image pixels, mimicking the actual behavior of the human brain's cognitive field of vision. Tang et al. trained a CNN fed with images generated from API Calls, obtaining an *accuracy* of **99%** [9]. Also, Ni et al. used convolutional networks to train images generated from opcodes, obtaining a maximum *accuracy* of **99.26%** and **98.86%** on average [10].

H. Convolutional Network Architectures

ResNetV2 is one of the most powerful convolutional network architectures today, it is a modified and improved version of ResNet [24] trained with the **Imagenet** [25] dataset. They differ mainly in the usage of *Batch Normalization* and *ReLU* activation function on the input before the convolution operation [26]. In particular, the architecture used in **(ResNet50V2)** consists of 50 hidden layers deep, hence the name. Ni et al. used a modified version of ResNet50v2 for malware detection using opcode images [10]. **MobileNetV2** is an architecture proposed in 2018, designed to build deep neural networks for mobile and embedded devices [27]. **MobileNetV2** is based on an optimized architecture that uses separable deep convolutions to reduce computational costs. Additionally, MobileNet was one of the models that had the best precision and robustness compared to models such as VGG16 [28], ResNet50, Xception [29] and a traditional Machine Learning model, XGBoost [30].

III. Dataset

The dataset used in this study is part of the work done by Oliveira [31]. This dataset includes data collected from 50,000 reports from *Cuckoo Sandbox Environment*. In this dataset, the first 100 non-consecutive API Calls were considered to avoid tracking loops, so from the 50,000 reports, 42,797 sequences of API Calls from malware and 1,079 sequences of API Calls from benign programs were chosen. Each API Call is assigned an integer number between 0 and 306 (there are 307 unique API Calls in the dataset.) Then, each sequence of API Calls in this dataset is structured such that the first column represents the hash of the sample, the next 100 columns contain an integer value between 0 and 306, which represents the corresponding API Call in the sequence; the last column contains a label for each sample, 0 for goodware and 1 for malware.

In order to counteract the class imbalance of this dataset, 1132 benign sequences obtained from *Cuckoo Sandbox Environment* reports were added. These reports were pulled from Reddit [32]. Still, this is not enough to counteract the imbalance and we are planning to look to other techniques such as undersampling in future work.

IV. Methodology

The methods used in this investigation were primarily two: Conversion of API calls into images and development of a CNN-based malware classifier, as Figure 1 shows.

A. Conversion of API Calls to Images

For the conversion of API calls to images, 2 methods were used: SimHash and FreqSeq. The following will explain how these methods were used in this research.

1) Simhash: SimHash method uses SHA-256 hashing algorithm. In order to apply it to the dataset the following steps where followed:

1) **Map API calls sequences codes to their corresponding names**

Authorized licensed use limited to: ESCUELA POLITECNICA DEL LITORAL (ESPOL). Downloaded on October 30,2023 at 03:27:43 UTC from IEEE Xplore. Restrictions apply.

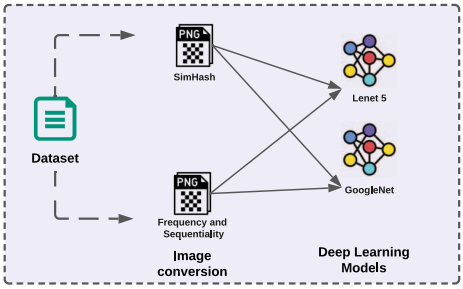

Figure 1. Methodology used to convert API Calls sequences to images and classify malware using Deep Learning.

a) Define a list with the API calls names within the dataset, assigning a number to each of them.
b) Read the .csv dataset file.
c) Extract API calls codes line by line.
d) Lookup for the corresponding API call name in the list that was created previously.

2) **Apply SimHash to the API calls sequences** In order to apply this method, it is necessary to choose a hashing algorithm. SHA-256 was chosen in this experiment because it produces sequences of 256 bits so we can arrange them in matrices of 16×16.
Algorithm 1 shows the SimHash (using SHA-256) pseudo-code.

Algorithm 1 SimHash using SHA256

```
Define a list of API Calls sequences
Initialize a zero vector v of size 256.
Initialize final vector f
for each seq in sequences do
    hash = sha_256(seq)
    for i=0 to len(hash) do
        if hash[i]==1 then
            v[i]+=1
        else
            v[i]-=1
        end if
    end for
end for
for i=0 to len(v) do
    if v[i] > 0 then
        f[i]=1
    else
        f[1]=0
    end if
end for
```

3) **Convert to image**
Lastly, the vector generated by algorithm 1 was converted into an image as follows:
a) Restructure the vector resulting from the previous step of 256 cells into a matrix of 16×16
b) Resize the number of pixels to the number needed according to the model to be trained. In this case the arrays were resized to 32×32 and 224×224 pixels.
c) Plot the binary matrix in a image of 16×16 cells using only black & white pixels.

4) **Image generation**
Finally, images of the hashes of all the 45008 sequences of API Calls will be generated by applying the SimHash method.

2) FreqSeq Algorithm (Frequency and Sequentiality): We developed a new algorithm to generate images based on the frequency and sequentiality of API calls. The steps used in this method are listed bellow:

FreqSeq Algorithm:

1) Give a different color to the 307 different API Calls that are available
2) Colorize a vector of 100 cells, giving each cell a color according to the API Call that corresponds to it from the sequence found in the dataset. (*Sequentiality*)
3) Alter the color intensity of each cell, depending on the frequency of the API Call that cell represents. (*Frequency*)
4) Transform the vector of 100 cells into a matrix of 10×10.
5) Convert matrix to image.

Finally, the steps were reproduced for each of the 45008 API call sequences within the dataset used for this research.

The sizes of the images generated with both SimHash and FreqSeq are not compatible with the size of the images required by the deep learning models to use. For this reason we resized all produced images to 32×32 and 224×224 pixels. The images generated by Simhash (16×16 pixels), could be modified without problems, since both 32 and 224 are multiples of 16. For the FreqSeq images, as they do not have dimensions that are multiples of 16. Thus, an interpolation method called *Nearest Neighbor Value Interpolation* was used to bring these images to the dimensions accepted by the models.

B. Application of CNNs on malware detection

In order to work with the generated images, two pre-trained CNN architectures were used: **ResNet50v2 and MobileNetV2** (**figure 2**), slightly modified in the output layer to be able to perform a binary classification. It should be noted that these architectures were chosen because of their popularity for image classification and because they are reduced versions of more complex architectures (greater number of hidden layers and therefore weights) such as **AlexNet** [33] or **VGG16** [28] but with the same (or better) percentage of effectiveness [34].

C. CNNs training

The training was carried out using the **Transfer Learning** technique in the mentioned architectures. Using transfer

Authorized licensed use limited to: ESCUELA POLITECNICA DEL LITORAL (ESPOL). Downloaded on October 30,2023 at 03:27:43 UTC from IEEE Xplore. Restrictions apply.

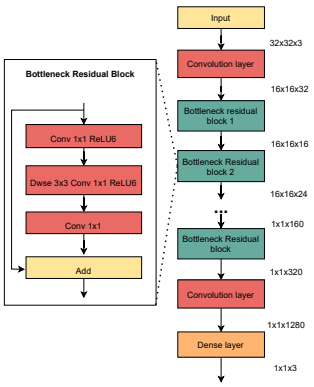

Figure 2. MobileNetV2 network architecture with an output layer with one unit (binary classification).

learning, the weights learned during the previous training of these networks are reused in large datasets specialized in the recognition and classification of images [35].

V. Results

This section presents the results of our experimental evaluation.

A. SimHash algorithm

In order to convert API calls into images using SimHash, the API calls codes were first mapped with its associated name, then the SimHash algorithm was applied on each API call sequence and produced a hash. Finally, this hash was transformed into an image.

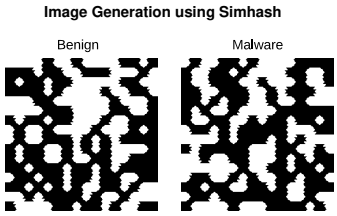

Figure 3. Images produced using SimHash.

B. FreqSeq Algorithm

When converting the API Calls into images using **FreqSeq**, the 307 API Calls were first given a different color. Then, matrices of 10×10 were generated, where each pixel represents the API Call in the sequence and the intensity of the color represents its frequency. These images considered the frequency and order of the API Calls in each sequence for both goodware and malware. Figure 4 shows the result of a benign sequence and a malware sequence.

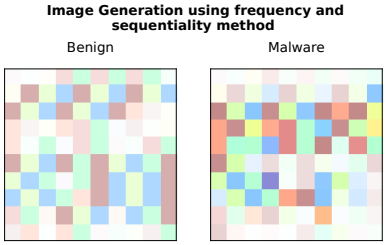

Figure 4. Images produced using FreqSeq.

C. ResNet50v2 training results

ResNet50v2 is a complex architecture in structure and number of parameters. Its training time is relatively long with 45 ms for each calculation performed per epoch among the more than 25 million internal parameters of the network.

In the first iteration of the training, the images generated by **Simhash** were fed to the network with dimensions of 32×32 pixels and 224×224 pixels; these being the minimum and maximum input sizes that it supports. Then, we retrained it with our **FreqSeq** images. As can be seen in **table I** the *F1 score* of the model varies very little between both types of images. However, it is clear that the best results belong to the 224×224 pixels images generated by **FreqSeq** with a score of **98.53%**.

Algorithm	Shape	% Acc.	% Prec.	% Recall	% F1
SimHash	32×32	96.75	97.30	99.21	98.21
	224×224	96.51	97.95	99.40	98.42
Freq. & Seq.	32×32	96.65	96.91	99.50	98.25
	$\mathbf{224 \times 224}$	97.30	98.00	99.51	**98.53**

Table I
ResNet50v2 performance metrics

D. MobileNetV2 training results on images produced by both algorihtms

MobileNetV2 is lighter, has fewer parameters and its training time is considerably less (56.76% faster) compared to *ResNet50v2*.

As with *ResNet50v2*, for the first training iteration we used **Simhash** images, both of 32×32 pixels and 224×224 pixels. Then, we repeated the training with the **FreqSeq** images.

This time, as can be seen in **table II**, the model's *F1 score* varies in favor of larger images (224 px), as it is higher for both types of images. As with the *ResNet50v2* architecture, the **FreqSeq** (224×224) option is the best performer, even slightly better than its ResNetV2 counterpart with a score of **98.57%**

VI. Discussion

We generated images from API Calls sequences in order to train models to identify malware. Two algorithms were used to convert API Call sequences to images, **Simhash** and **FreqSeq**. The results were compared by training two well-known Convolutional Network architectures (**ResNet50v2 and MobileNetv2**).

Authorized licensed use limited to: ESCUELA POLITECNICA DEL LITORAL (ESPOL). Downloaded on October 30,2023 at 03:27:43 UTC from IEEE Xplore. Restrictions apply.

Algorithm	Shape	% Acc.	% Prec.	% Recall	% F1
SimHash	32×32	98.60	95.90	99.65	97.72
	224×224	98.50	97.58	99.86	98.52
Freq. & Seq.	32×32	95.80	96.15	99.55	97.80
	$\mathbf{224 \times 224}$	97.25	97.70	99.45	**98.57**

Table II
MOBILENETV2 PERFORMANCE METRICS

Our results show that the use of images for malware detection generates results with high accuracy. **MobileNetV2** presented better results based reaching a precision of 98.60% when analyzing Simhash images of 32×32 pixels; compared to **ResNet50v2** that obtained 96.75% for Simhash and 97.30% for FreqSeq. Now, due to class imbalance, the F_1-score metric is considered a better performance indicator. In this case, **MobileNetV2** presented the best F_1-score, obtaining a value of 98.57% analyzing FreqSeq images of 224×224 pixels in contrast to **ResNet50v2**'s score of 98.53%. However, on average, the architecture with the best F_1-score was **ResNet50v2** with 98.3525% compared to **MobileNetV2** with 98.1525%.

This research obtained results very similar to those of Ni et al. [10], a maximum average F_1-score of 98.07%. Note that their F_1-score was obtained with Simhash combining SHA-256 and SHA-512 hashes. However, we observed an average F_1-score of 98.2175% for Simhash using just SHA-256.

Class imbalance is one of our main drawbacks. This is a common problem in datasets used for classification. For this case, the original dataset contained 42,797 malware sequences compared to 1,079 benign sequences. This creates a bias for the model to identify the majority of sequences as malware. To counteract class imbalance, we included benign sequences from another dataset increasing this number to 2211 sequences; still just a marginal increase. This is why we rather use the F_1-score metric to compare the performance of our models.

The size of the generated images was another problem. Simhash (using SHA-256) generated images of 16×16px, which were simple to resize to 32×32px. On the other hand, images generated with FreqSeq were of size 10×10px. So, we use *Nearest Neighbor* interpolation to bring them to the required size. It is possible this could influence the results and cause FreqSeq to get a better F_1-score than Simhash. We will continue analyzing this as we move forward with our research.

VII. CONCLUSIONS AND FUTURE WORK

We obtained satisfactory results during our research. Regarding the algorithms that were used for the conversion of sequences of API calls to images, **FreqSeq** showed slightly better results. However, it is important to note that the size of the images influences the precision of the model. Images with a larger size (224×224 px) had, on average, a better percentage of precision in both models.

As far as the CNN architecture is concerned, the **ResNet50v2** obtained a better F_1-score on average (98.35%) compared to **MobileNetV2** (98.15%) considering both types of images.

This research opens the path for us to continue exploring image conversion algorithms and improve the efficiency and speed of conversion, with the purpose of applying it in early detection systems such as IDS. Besides trying to reduce the imbalance in out dataset, we also wish to experiment with more recent architectures in the field of Computer Vision such as **Transformers**.

REFERENCES

[1] R. Sobers, "Sponsored: 81 ransomware statistics, data, trends and facts for 2021," Sep 2021.

[2] H. S. Lallie, L. A. Shepherd, J. R. Nurse, A. Erola, G. Epiphaniou, C. Maple, and X. Bellekens, "Cyber security in the age of covid-19: A timeline and analysis of cyber-crime and cyber-attacks during the pandemic," *Computers & Security*, vol. 105, p. 102248, 2021.

[3] H. Diazgranados and L. Grustniy, "Ciberataques en américa latina crecen un 24% durante los primeros ocho meses de 2021."

[4] B. Pranggono and A. Arabo, "Covid -19 pandemic cybersecurity issues," *Internet Technology Letters*, vol. 4, no. 2, 2020.

[5] P. Okane, S. Sezer, and K. Mclaughlin, "Obfuscation: The hidden malware," *IEEE Security & Privacy*, vol. 9, no. 5, p. 41–47, 2011.

[6] R. Sihwail, K. Omar, and K. A. Z. Ariffin, "An effective memory analysis for malware detection and classification," *Computers, Materials & Continua*, vol. 67, no. 2, p. 2301–2320, 2021.

[7] G. G. Sundarkumar, V. Ravi, I. Nwogu, and V. Govindaraju, "Malware detection via api calls, topic models and machine learning," in *2015 IEEE International Conference on Automation Science and Engineering (CASE)*, pp. 1212–1217, IEEE, 2015.

[8] A. Souri and R. Hosseini, "A state-of-the-art survey of malware detection approaches using data mining techniques," *Human-centric Computing and Information Sciences*, vol. 8, no. 1, 2018.

[9] M. Tang and Q. Qian, "Dynamic api call sequence visualisation for malware classification," *IET Information Security*, vol. 13, no. 4, pp. 367–377, 2019.

[10] S. Ni, Q. Qian, and R. Zhang, "Malware identification using visualization images and deep learning," *Computers & Security*, vol. 77, p. 871–885, 2018.

[11] C. Li and J. Zheng, "Api call-based malware classification using recurrent neural networks," *Journal of Cyber Security and Mobility*, pp. 617–640, 2021.

[12] A. S. Bozkir, E. Tahillioglu, M. Aydos, and I. Kara, "Catch them alive: A malware detection approach through memory forensics, manifold learning and computer vision," *Computers & Security*, vol. 103, p. 102166, 2021.

[13] A. Schranko de Oliveira and R. J. Sassi, "Behavioral malware detection using deep graph convolutional neural networks," 2019.

[14] R. Sihwail, K. Omar, and K. A. Zainol Ariffin, "A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis," vol. 8, p. 1662, 09 2018.

[15] I. You and K. Yim, "Malware obfuscation techniques: A brief survey," in *2010 International conference on broadband, wireless computing, communication and applications*, pp. 297–300, IEEE, 2010.

[16] M. I. Sharif, A. Lanzi, J. T. Giffin, and W. Lee, "Impeding malware analysis using conditional code obfuscation.," in *NDSS*, Citeseer, 2008.

[17] R. Mosli, R. Li, B. Yuan, and Y. Pan, "Automated malware detection using artifacts in forensic memory images," in *2016 IEEE Symposium on Technologies for Homeland Security (HST)*, pp. 1–6, IEEE, 2016.

[18] S. Banin, A. Shalaginov, and K. Franke, "Memory access patterns for malware detection," 2016.

[19] Y. Dai, H. Li, Y. Qian, and X. Lu, "A malware classification method based on memory dump grayscale image," *Digital Investigation*, vol. 27, pp. 30–37g, 2018.

[20] N. Hampton, Z. Baig, and S. Zeadally, "Ransomware behavioural analysis on windows platforms," *Journal of information security and applications*, vol. 40, pp. 44–51, 2018.

[21] Y. Ki, E. Kim, and H. K. Kim, "A novel approach to detect malware based on api call sequence analysis," *International Journal of Distributed Sensor Networks*, vol. 11, no. 6, p. 659101, 2015.

[22] S. Buyrukbilen and S. Bakiras, "Secure similar document detection with simhash," in *Workshop on Secure Data Management*, pp. 61–75, Springer, 2013.

Authorized licensed use limited to: ESCUELA POLITECNICA DEL LITORAL (ESPOL). Downloaded on October 30,2023 at 03:27:43 UTC from IEEE Xplore. Restrictions apply.

[23] Y. Lecun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, p. 436–444, 2015.
[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015.
[25] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009.
[26] M. Rahimzadeh and A. Attar, "A modified deep convolutional neural network for detecting covid-19 and pneumonia from chest x-ray images based on the concatenation of xception and resnet50v2," *Informatics in Medicine Unlocked*, vol. 19, p. 100360, 2020.
[27] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," 2019.
[28] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015.
[29] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," July 2017.
[30] V. Carletti, A. Greco, A. Saggese, and M. Vento, "Robustness evaluation of convolutional neural networks for malware classification," 2021.
[31] A. Oliveira, "Malware analysis datasets: Api call sequences," 2019.
[32] L. Zenóbio, "Malware and benign windows PE cuckoo reports," 2020.
[33] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," vol. 25, 2012.
[34] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," 2014.
[35] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

Authorized licensed use limited to: ESCUELA POLITECNICA DEL LITORAL (ESPOL). Downloaded on October 30,2023 at 03:27:43 UTC from IEEE Xplore. Restrictions apply.